

Assessment Tool 2 – Technical Presentation Answer (CO5)

Topic 3: Utility-Based Agent for E-Commerce Recommendations

1. Technical Content — What is a Utility-Based Agent?

A **utility-based agent** does not just pick any action that achieves a goal (like a goal-based agent) — it evaluates **multiple possible actions/options**, assigns each a **utility value** (a numeric measure of “how good” that outcome is), and selects the action that **maximises expected utility**. This is essential for e-commerce recommendations because there are usually many products that could satisfy a user’s need, and the agent must rank them by how well each serves multiple, sometimes competing, objectives (relevance, price, popularity, delivery time, seller rating, profit margin).

Utility function example:

$$\begin{aligned}\text{Utility}(\text{product}) = & w1 * \text{Relevance}(\text{user}, \text{product}) \\ & + w2 * \text{Price_fit}(\text{user_budget}, \text{price}) \\ & + w3 * \text{Popularity}(\text{product}) \\ & + w4 * \text{Rating}(\text{product}) \\ & + w5 * \text{Margin}(\text{product})\end{aligned}$$

where $w1..w5$ are tunable weights reflecting business priorities, learned/tuned over time from user engagement and conversion feedback.

2. Problem Identification

E-commerce platforms face: (a) massive product catalogs (millions of SKUs), (b) millions of concurrent users with different preferences, (c) the need to balance user satisfaction against business goals (margin, inventory clearance), and (d) real-time response expectations (recommendations must render in milliseconds). A simple content-based or collaborative-filter score alone doesn’t capture this multi-objective trade-off — hence the need for a utility-based agent architecture that explicitly models and optimises a weighted objective across candidate items, computed at scale using distributed data processing.

3. System Design & Use of Tools/Data

Pipeline: 1. **Candidate Generation (PySpark RDD/DataFrame):** for each user, generate a candidate pool of a few hundred products using collaborative filtering (e.g., Spark MLlib ALS) and content similarity (map/join on category/attribute RDDs) — reduces the full catalog to a manageable candidate set per user. 2. **Feature Computation (mapPartitions):** compute per-candidate features in parallel across partitions — relevance score, price-fit, popularity (from aggregated click/purchase RDDs via

reduceByKey), rating, margin. 3. **Utility Scoring (Agent Layer)**: the utility-based agent applies the weighted utility function to rank candidates per user; this can run as a Spark UDF over the candidate RDD/DataFrame for parallel scoring across all users simultaneously. 4. **Top-K Selection (takeOrdered / top)**: select the top-K highest-utility products per user to display. 5. **Serving Layer**: results cached in a low-latency store (Redis) for real-time page rendering.

Large-scale processing: with millions of users × hundreds of candidates each, this is a large distributed computation; Spark's partitioning (by user_id) allows the utility scoring for different users to run fully in parallel across cluster nodes, and reduceByKey/aggregateByKey operations efficiently compute the popularity/behaviour statistics feeding the utility function from raw clickstream logs at scale.

4. Communication — Presentation Structure (suggested slide flow)

1. Title & problem statement (why simple recommenders aren't enough)
2. What is a utility-based agent (PEAS description: Performance measure = expected utility/engagement, Environment = catalog + user session, Actuators = displayed recommendations, Sensors = clickstream/purchase events)
3. Utility function design and weight tuning
4. Distributed architecture diagram (candidate generation → feature computation → utility scoring → serving)
5. PySpark RDD operations used at each stage
6. Example: how the utility score changes the ranking vs. plain relevance-only ranking
7. Optimizing user satisfaction: A/B testing utility weights, feedback loop retraining the weights/relevance model
8. Conclusion & future extensions (adding a learning agent to auto-tune weights)

5. Professionalism / Ethical Note

Utility functions must avoid manipulative optimization purely for margin at the expense of user trust (e.g., always pushing high-margin items regardless of fit) — a well-designed agent should be transparent about the objectives it balances and be periodically audited for fairness (e.g., not systematically disadvantaging lower-spend users with worse recommendations).

6. Summary

The Utility-Based Agent for e-commerce recommendation ranks candidate products by a multi-factor utility score rather than a single relevance metric, computed at scale via PySpark's distributed RDD/DataFrame operations (map, reduceByKey, mapPartitions, takeOrdered), enabling personalized, business-aligned recommendations for large user bases in real time.